

# JavaScript From Zero to Minecraft

A practical beginner's book for the Programming Gamer

Variables • Conditions • Loops • Functions • Arrays • Objects • `forEach()`

## How to Use This Book

Programming ka goal sirf code yaad karna nahi hai. Har chapter mein pehle idea samjho, phir example dekho, phir khud se code likho. Examples Minecraft-style hain taaki concepts practical feel hon.

Semicolon (;) JavaScript mein aksar optional hota hai because of Automatic Semicolon Insertion, lekin consistent style ke liye semicolon lagana achhi habit hai.

## 1. Variables: `let` and `const`

Variable ek named container jaisa hai jisme value store hoti hai. `let` ko reassign kar sakte ho; `const` ko reassign nahi kar sakte.

```
let diamonds = 42;
console.log(diamonds);

diamonds = 50;
console.log(diamonds);

const maxHealth = 20;
// maxHealth = 30; // Error
```

## 2. `console.log()`

`console.log()` console/output mein information print karta hai.

```
let player = "Steve";
console.log(player);
console.log("Welcome!");
```

## 3. Numbers and Operators

Common operators: `+`, `-`, `*`, `/`, `%`, `++`, `--`, `+=`, `-=`, `*=`, `/=`

```
let health = 20;
health += 5;
health -= 3;
health *= 2;
health /= 2;

console.log(health);
```

## 4. Strings and Template Literals

String text hota hai. Template literal backticks (```) use karta hai aur `${...}` ke through values insert karta hai.

```
let player = "Steve";
let diamonds = 42;

console.log(`Welcome ${player}!`);
console.log(`${player} has ${diamonds} diamonds.`);
```

## 5. Comparisons and Booleans

Comparison expressions true ya false return karte hain. Important: = assignment hai; === comparison hai.

```
let health = 20;

health = 10;      // value change
console.log(health === 10); // comparison
```

## 6. if / else / else if

Conditions program ko decision lene deti hain.

```
let health = 7;

if (health < 10) {
  console.log("Low health");
} else {
  console.log("Health is fine");
}

let diamonds = 42;

if (diamonds >= 50) {
  console.log("Rich");
} else {
  console.log("Keep mining");
}
```

## 7. Logical Operators

&& = dono true. || = kam se kam ek true. ! = boolean ka opposite.

```
let health = 20;
let hasSword = true;

if (health > 0 && hasSword) {
  console.log("Ready to fight");
}

let hasBow = true;

if (hasSword || hasBow) {
  console.log("Has a weapon");
}

let dead = false;
console.log(!dead);
```

## 8. for Loops

for loop repeated work ke liye useful hai. Basic structure: start; condition; update.

```
for (let i = 1; i <= 10; i++) {
  console.log(i);
}

let tools = ["Sword", "Pickaxe", "Axe"];

for (let i = 0; i < tools.length; i++) {
  console.log(tools[i]);
}
```

## 9. Functions

Function reusable code block hai. Parameter input deta hai; return result wapas deta hai.

```
function double(number) {
```

```

    return number * 2;
}

console.log(double(7));

function add(a, b) {
    return a + b;
}

console.log(add(8, 5));

```

## 10. Arrays

Array ordered list hai. Index 0 se start hota hai. push() end mein add karta hai; pop() last item remove karta hai.

```

let players = ["Steve", "Alex", "Shahid"];

console.log(players[0]);
console.log(players.length);

players[1] = "Herobrine";
players.push("Alex");
players.pop();

```

## 11. Objects

Object related properties ko key-value pairs mein organize karta hai.

```

let player = {
    name: "Steve",
    health: 20,
    diamonds: 42
};

console.log(player.name);
player.health = 15;
player.level = 10;
delete player.diamonds;

```

## 12. Arrays of Objects

Array ke andar multiple objects rakh sakte ho.

```

let players = [
    { name: "Steve", health: 20 },
    { name: "Alex", health: 18 },
    { name: "Shahid", health: 15 }
];

console.log(players[0].name);
console.log(players[1].health);

```

## 13. Arrays + Objects + Loops

players[i] current object hai; players[i].name us object ki property hai.

```

let players = [
    { name: "Steve", health: 20 },
    { name: "Alex", health: 18 },
    { name: "Shahid", health: 15 }
];

for (let i = 0; i < players.length; i++) {
    console.log(players[i].name);
    console.log(players[i].health);
}

```

```
}
```

## 14. forEach()

forEach() array ke har element ke liye diya gaya function ek baar chalata hai.

```
let items = ["Sword", "Pickaxe", "Shield"];

items.forEach(function(item) {
  console.log(item);
});

let players = ["Steve", "Alex", "Shahid"];

players.forEach(function(player, index) {
  console.log(`${index}: ${player}`);
});
```

## 15. for vs forEach

for loop mein index aur loop condition par direct control hota hai. forEach() mein array automatically har element ke liye function chalata hai.

```
// for
for (let i = 0; i < players.length; i++) {
  console.log(players[i]);
}

// forEach
players.forEach(function(player) {
  console.log(player);
});
```

## 16. Common Mistakes

- = aur === ko confuse mat karo.
- Array indexing 0 se start hoti hai.
- push/pop arrays ke methods hain; object property ke liye assignment use karo.
- console.log() print karta hai; return value function se bahar deta hai.
- Loop mein array.length useful hai.
- Variables ko generally let/const se declare karo.

## 17. Minecraft Thinking Patterns

Problem ko data + logic + repetition mein todna seekho. Ye thinking baad mein Minecraft Bedrock Script API ke saath connect hogi.

```
let player = {
  name: "Steve",
  health: 20,
  diamonds: 42
};

if (player.health <= 0) {
  console.log(`${player.name} is dead`);
}

function heal(player, amount) {
  player.health += amount;
  return player.health;
}

heal(player, 5);
```

## 18. Quick Reference

Core mental model: Data → variables / arrays / objects Decision → if / else Repetition → for / forEach Reuse → functions Output → console.log() Result → return

Next lessons mein naye concepts one-by-one aayenge, aur beech-beech mein sirf already-learned concepts ka mixed practice hoga.